API Systems - Task 3 Critical Evaluation

Overview

This project involved designing, developing and testing a RESTful API for FlashPoint MT, a journalistic mobile application for Maltese citizens built around an interactive map where users can discover and verify local news. My responsibilities covered the authentication system, the users resource, the memberships resource, and some of the Ionic Angular frontend infrastructure mainly the profile page and media pages. Across three sprints spanning several weeks. Looking back on the full process, there is a lot to reflect on in terms of what was planned, what caused the most difficulty, and what would be done differently with more time.

Design and Planning

The planning phase began in Sprint 1 with setting up the development environment. We discussed our idea throughout two other units Entrepreneurship and Prototyping and testing. We also mapped out how the website itself will look like and function. I installed Node.js using terminal on my windows and then installing the Angular and Ionic seemed straightforward but the first attempt at running ionic serve failed. This was a minor issue but I managed to update the software and looked up some tutorials, and it worked fine.

The API was built around a central router in index.php which would take in all the requests and send them to the right place depending on the URL. The idea behind this was to keep everything organised in one place rather than having random separate files handling different endpoints. The router worked by using a regex to clean up the URL and pull out just the API path, which was fine in theory but caused a really annoying issue early on because I had not accounted for the parent folder API_Systems_Nirvana_Yan being part of the path. So every single request was coming back as not found even though the server was running perfectly fine, which made it very confusing to debug because from the outside everything looked like it should be working.

Development Process

The development of the application was a pretty long process overall. Me and Nirvana split the work between us where she handled most of the frontend while I focused on the backend side, specifically the login, registration, account deletion, memberships and the users resource. We also each had our own sections of the frontend to work on which meant a lot of back and forth between our laptops to keep things in sync.

The registration endpoint was the first real thing I built and it caused quite a few issues straight away. My first attempt used uniqid() to generate IDs for users which immediately failed with a foreign key constraint error because the database was using integer auto increment columns, not strings. Switching to lastInsertId() sorted that out but it was a good reminder to check the schema before writing any insert logic. On top of that I had typed users_id instead of user_id for the memberships table column which caused another error. Unfortunately it took way too long too figure out what was causing the issues but managed.

I also wrapped all three database operations involved in registration inside a PDO transaction after realising what was happening without one. If the membership insert failed, a user row would get left sitting in the database with nothing linked to it. The transaction made sure all three operations either went through together or rolled back cleanly which is just the safer way to handle anything touching multiple tables at once.

The login endpoint was where I built the JWT token system. The token gets signed using HMAC-SHA256 with a secret key and the payload carries the user ID, the display name, the role and an expiry timestamp set to one hour. I built this from scratch rather than using a library.

The users end ended up being the most involved part of the backend, covering seven endpoints including get profile, get membership with all the feature flags from the membershipTiers table, upgrade or downgrade membership with logging to the in_app_purchases table, cancel subscription, change password and photo upload. One bug that caught me off guard was a PHP match() statement I was using for routing inside handleUsers() which was printing the return value of 1 straight into the response body instead of actually calling the function. Replacing it with if/else blocks fixed it but it was tricky to spot at first because the response still looked almost right.

At first when we merged our work there were not any major clashes but some things did break when it came to the endpoints, the login being one of them. After fixing that I carried on with the memberships and other endpoints but once we merged again there was more breakage in the app. We eventually figured out that we had differences in our databases and also in the path files, where I was using helpers.php and Nirvana was using config.php, and the two were not compatible with each other. I then went through and merged both databases together and unified the API paths into a single shared config.php that auto-detects whether the server is running on XAMPP or MAMP, so both of us could work on the same codebase without changing anything between machines.

On the frontend side I worked on some of the login page UI, adding the show/hide password toggle using Angular property binding, the Facebook, Google and Twitter social login buttons

using inline SVG, and the full auth.ts service wiring the login form up to the PHP API through HttpClient. After connecting the auth service a routing error came up, NG04002 Cannot match any routes, which took a while to pin down. It turned out there was a duplicate tabs path in tabs.routes.ts conflicting with the parent route. Removing the duplicate and changing the child path to an empty string fixed it.

Testing via Postman

Testing was done consistently across all three sprints with Postman. For every endpoint I tested both the success cases and the error cases in collections split across Auth, Users and Memberships.

The most frustrating issue throughout the whole project was the Authorization header not getting passed through by XAMPP on Windows. I was sending the Bearer token correctly in Postman but the API kept coming back with 401 no token provided. After a lot of digging it turned out Apache on XAMPP blocks the Authorization header by default and does not pass it to PHP through the normal HTTP_AUTHORIZATION server variable. The fix was adding two lines to the .htaccess file to force Apache to pass it through, and also updating verifyToken() to check three different places in order. The reason it took so long is that the token was being sent correctly the whole time, it was just being silently dropped before PHP ever saw it, so from the outside it looked like a token problem when it was actually a server configuration problem.

Results and Reflection

By the end the API had a working authentication system with register and login, a full users resource across seven endpoints, a memberships resource with get and upgrade functionality, and all of it connected to the Ionic Angular frontend.

Looking through our project development, the biggest source of problems was environment-specific behaviour that was hard to predict. The XAMPP Authorization header issue, the match() statement printing to the response and the nested folder routing regex were all in that category. None of them were bugs in the logic itself, they were things that only showed up in a specific server setup or PHP version which made them much harder to figure out in fact I believe even the lecturer was encountering this issue. If I had spent more time going through the XAMPP and Apache documentation before starting I probably would have caught the htaccess issue before it burned so much time.

Working alongside Nirvana also showed how important it is to agree on conventions before anyone writes a line of code. The sub versus id mismatch in the token payload and the different column names in the articles table both caused issues that had to be fixed later on after loads of time of debugging and debriefing.

Future Improvements

There are a few things that would make the API noticeably better with more time. The social login buttons are there on the login page but they are not actually connected to anything. Getting the Google, Facebook and Twitter OAuth 2.0 flows properly implemented was in the original plan from Task 1 and it would make a real difference to how easy it is to sign up.

I would also focus more on improving the front end and make it more aesthetic and as close to our branding plan as possible. I also believe that the endpoints need more refinement and if I had more time I would go through the entire code again to see what differences there are and officially merge everything and fix all the things that were breaking in the app.

Another thing I would improve is moving the token out of localStorage and into httpOnly cookies would be a worthwhile security improvement too. localStorage can be read by any JavaScript on the page which through some research I did, is a known vulnerability, whereas httpOnly cookies are completely inaccessible to scripts. It would mean updating both the PHP API and the Angular service but it would make the authentication system considerably more secure.